



C.A.S.A. Course

CONTEXT AWARE SECURITY ANALYTICS IN COMPUTER VISION

proff. Michele Nappi, Fabio Narducci and Carmen Bisogni

aa.2024/2025



Tools



Git and GitHub



Anaconda ... & friends



Python 3.6



UNIVERSITÀ
DEGLI STUDI
DI SALERNO



Git



- is a command-line tool used for version control;
- runs locally on a computer;
- is completely self-sufficient;
- it does not require the use of any external, cloud hosting service.

Most used Git command-line tools:

- commit
- push
- pull
- revert
- ...



UNIVERSITÀ
DEGLI STUDI
DI SALERNO



Git command-line tools



git config	
git config --global user.name <name>	Define the author name to be used for all commits by the current user.
git config --global user.email <email>	Define the author email to be used for all commits by the current user.
git config --global alias. <alias-name> <git-command>	Create shortcut for a Git command. E.g. alias.glog log --graph --oneline will set git glog equivalent to git log --graph --oneline.
git config --system core.editor <editor>	Set text editor used by commands for all users on the machine. <editor> arg should be the command that launches the desired editor (e.g., vi).
git config --global --edit	Open the global configuration file in a text editor for manual editing.
git log	
git log --<limit>	Limit number of commits by <limit>. E.g. git log -5 will limit to 5 commits.
git log --oneline	Condense each commit to a single line.
git log -p	Display the full diff of each commit.
git log --stat	Include which files were altered and the relative number of lines that were added or deleted from each of them.
git log --author="<pattern>"	Search for commits by a particular author.
git log --grep="<pattern>"	Search for commits with a commit message that matches <pattern>.
git log <since>..<until>	Show commits that occur between <since> and <until>. Args can be a commit ID, branch name, HEAD, or any other kind of revision reference.
git log -- <file>	Only display commits that have the specified file.
git log --graph --decorate	--graph flag draws a text based graph of commits on left side of commit msgs. --decorate adds names of branches or tags of commits shown.
git diff	
git diff HEAD	Show difference between working directory and last commit.
git diff --cached	Show difference between staged changes and last commit
git reset	
git reset	Reset staging area to match most recent commit, but leave the working directory unchanged.
git reset --hard	Reset staging area and working directory to match most recent commit and overwrites all changes in the working directory.
git reset <commit>	Move the current branch tip backward to <commit>, reset the staging area to match, but leave the working directory alone.
git reset --hard <commit>	Same as previous, but resets both the staging area & working directory to match. Deletes uncommitted changes, and all commits after <commit>.
git rebase	
git rebase -i <base>	Interactively rebase current branch onto <base>. Launches editor to enter commands for how each commit will be transferred to the new base.
git pull	
git pull --rebase <remote>	Fetch the remote's copy of current branch and rebases it into the local copy. Uses git rebase instead of merge to integrate the branches.
git push	
git push <remote> --force	Forces the git push even if it results in a non-fast-forward merge. Do not use the --force flag unless you're absolutely sure you know what you're doing.
git push <remote> --all	Push all of your local branches to the specified remote.
git push <remote> --tags	Tags aren't automatically pushed when you push a branch or use the --all flag. The --tags flag sends all of your local tags to the remote repo.



UNIVERSITÀ
DEGLI STUDI
DI SALERNO



Git command-line tools



Git Basics		Rewriting Git History	
git init <directory>	Create empty Git repo in specified directory. Run with no arguments to initialize the current directory as a git repository.	git commit --amend	Replace the last commit with the staged changes and last commit combined. Use with nothing staged to edit the last commit's message.
git clone <repo>	Clone repo located at <repo> onto local machine. Original repo can be located on the local filesystem or on a remote machine via HTTP or SSH.	git rebase <base>	Rebase the current branch onto <base>. <base> can be a commit ID, a branch name, a tag, or a relative reference to HEAD.
git config user.name <name>	Define author name to be used for all commits in current repo. Devs commonly use --global flag to set config options for current user.	git reflog	Show a log of changes to the local repository's HEAD. Add --relative-date flag to show date info or --all to show all refs.
git add <directory>	Stage all changes in <directory> for the next commit. Replace <directory> with a <file> to change a specific file.	Git Branches	
git commit -m "<message>"	Commit the staged snapshot, but instead of launching a text editor, use <message> as the commit message.	git branch	List all of the branches in your repo. Add a <branch> argument to create a new branch with the name <branch>.
git status	List which files are staged, unstaged, and untracked.	git checkout -b <branch>	Create and check out a new branch named <branch>. Drop the -b flag to checkout an existing branch.
git log	Display the entire commit history using the default format. For customization see additional options.	git merge <branch>	Merge <branch> into the current branch.
git diff	Show unstaged changes between your index and working directory.	Remote Repositories	
Undoing Changes		git remote add <name> <url>	Create a new connection to a remote repo. After adding a remote, you can use <name> as a shortcut for <url> in other commands.
git revert <commit>	Create new commit that undoes all of the changes made in <commit>, then apply it to the current branch.	git fetch <remote> <branch>	Fetches a specific <branch>, from the repo. Leave off <branch> to fetch all remote refs.
git reset <file>	Remove <file> from the staging area, but leave the working directory unchanged. This unstages a file without overwriting any changes.	git pull <remote>	Fetch the specified remote's copy of current branch and immediately merge it into the local copy.
git clean -n	Shows which files would be removed from working directory. Use the -f flag in place of the -n flag to execute the clean.	git push <remote> <branch>	Push the branch to <remote>, along with necessary commits and objects. Creates named branch in the remote repo if it doesn't exist.



UNIVERSITÀ
DEGLI STUDI
DI SALERNO



GitHub



- is a cloud-based platform built around the Git tool;
- online editing of file;
- is an online service to which developers who use Git can connect and upload or download resources.



UNIVERSITÀ
DEGLI STUDI
DI SALERNO



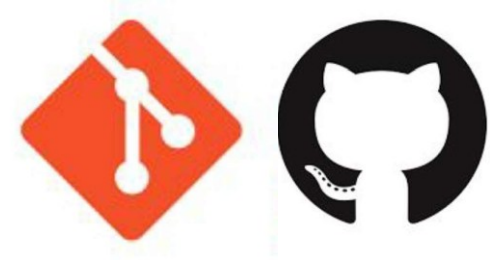


Git vs. GitHub comparision



GIT	GITHUB
Installed locally	Hosted in the cloud
First released in 2005	Company launched in 2008
Maintained by The Linux Foundation	Purchased in 2018 by Microsoft
Focused on version control and code sharing	Focused on centralized source code hosting
Primarily a command-line tool	Administered through the web
Provides a desktop interface named Git Gui	Desktop interface named GitHub Desktop
No user management features	Built-in user management
Minimal external tool configuration features	Active marketplace for tool integration
Competes with Mercurial, Subversion, IBM, Rational Team Concert and ClearCase	Competes with Atlassian Bitbucket and GitLab
Open source licensed	Includes a free tier and pay-for-use tiers

Git & GitHub



As you can see, Git and GitHub are clearly different in features and purpose

- they compliment each other
- they are not competitors

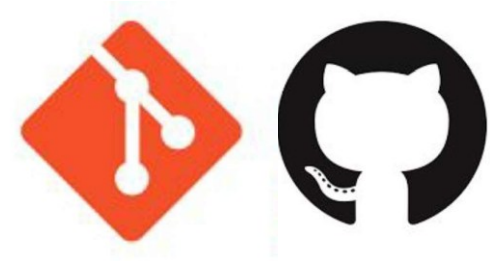
Integrating Git and GitHub together is a powerful combination.



UNIVERSITÀ
DEGLI STUDI
DI SALERNO



Let's start



- 1) Set up a GitHub account <https://github.com>
- 2) Download GitHub desktop on you computer <https://desktop.github.com>
- 3) Create a repository in local and an URL will uniquely identifies it.
- 4) With the repository created, developers can install Git locally and **clone** the repository.
- 5) Developers can make local changes to project files and then **push** the **commit** back to GitHub.
- 6) To get updates developers use **pull** and **fetch** commands.



UNIVERSITÀ
DEGLI STUDI
DI SALERNO

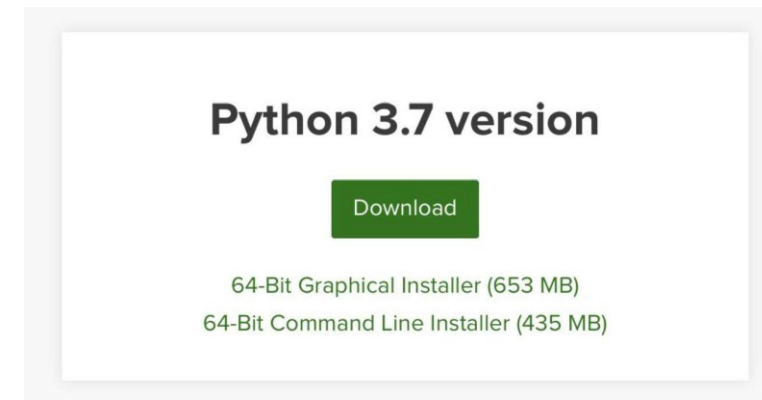




The open-source **Anaconda Distribution** is the easiest way to perform Python/R data science and machine learning on Linux, Windows, and Mac OS X. With over 15 million users worldwide, it is the industry standard for developing, testing, and training on a single machine, enabling *individual data scientists* to:

- Quickly download 1,500+ Python/R data science packages
- Manage libraries, dependencies, and environments with **Conda**
- Develop and train machine learning and deep learning models with **scikit-learn**, **TensorFlow**, and **Theano**
- Analyze data with scalability and performance with **Dask**, **NumPy**, **pandas**, and **Numba**
- Visualize results with **Matplotlib**, **Bokeh**, **Datashader**, and **Holoviews**

Download Anaconda on
www.anaconda.com



UNIVERSITÀ
DEGLI STUDI
DI SALERNO



Do not install Python 2.7

```
Using TensorFlow backend.
```

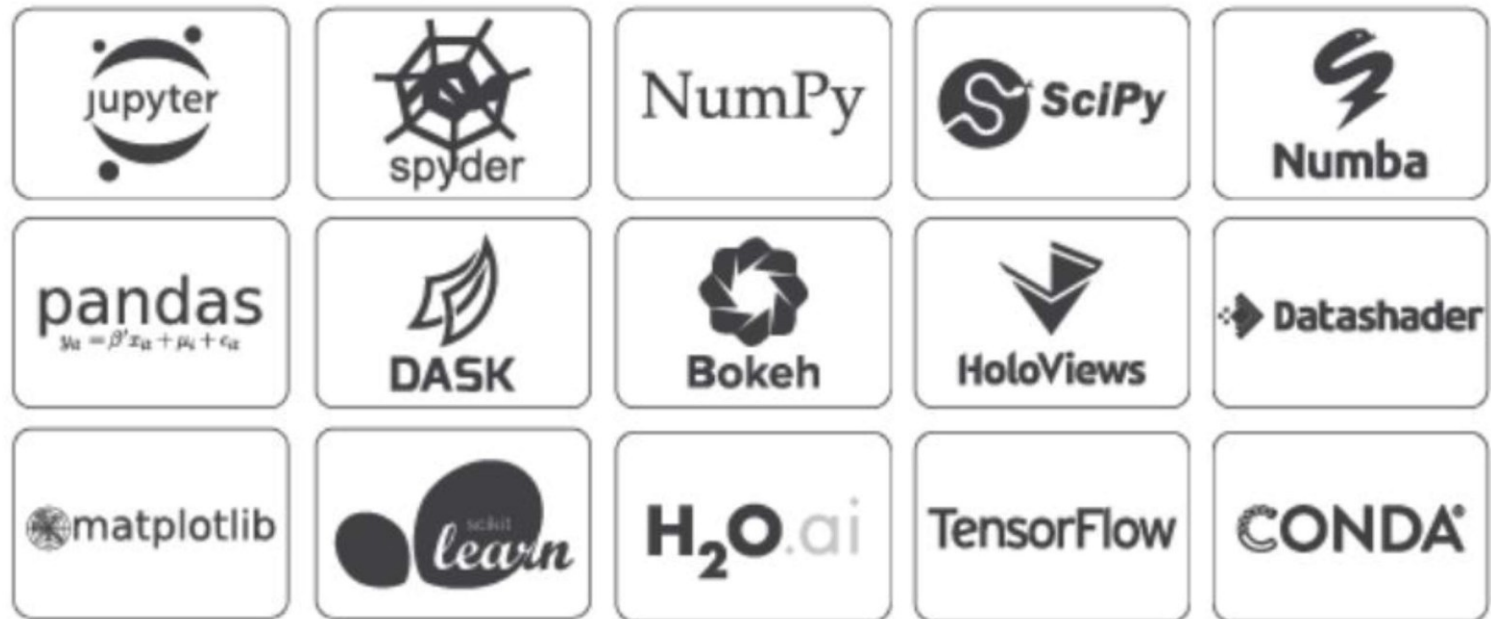
```
DEPRECATION: Python 2.7 will reach the end of its life on January 1st, 2020  
. Please upgrade your Python as Python 2.7 won't be maintained after that d  
ate. A future version of pip will drop support for Python 2.7.
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python2.7/site-pack  
age (1.5.6)
```



UNIVERSITÀ
DEGLI STUDI
DI SALERNO





UNIVERSITÀ
DEGLI STUDI
DI SALERNO



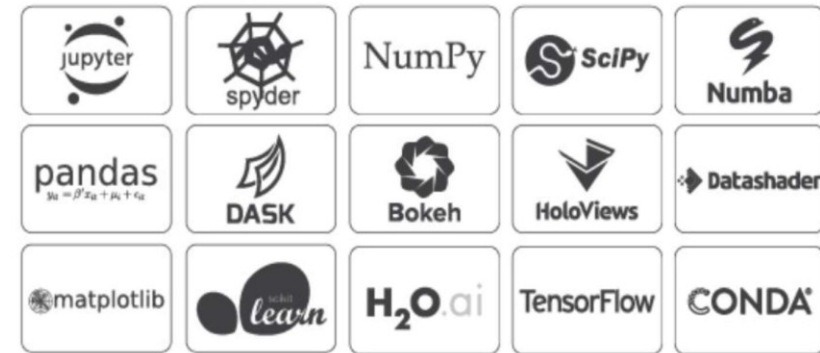
Conda



Package, dependency and environment management for any language—Python, R, Ruby, Lua, Scala, Java, JavaScript, C/ C++, FORTRAN, and more.

Conda as a package manager helps you find and install packages. If you need a package that requires a different version of Python, you do not need to switch to a different environment manager, because conda is also an environment manager. With just a few commands, you can set up a totally separate environment to run that different version of Python, while continuing to run your usual version of Python in your normal environment.

And a lot of other things...



scikit 1



UNIVERSITÀ
DEGLI STUDI
DI SALERNO



Create a conda environment



#Create a new environment

conda create -n name_env python=3.7

conda activate name_env

conda deactivate

#Enter your new environment

#Exit all the environments

matplotlib



Is the best plotting library out there. Install it:

conda install -c conda-forge matplotlib



UNIVERSITÀ
DEGLI STUDI
DI SALERNO



scikit-learn



Is the best Machine Learning library in the world.

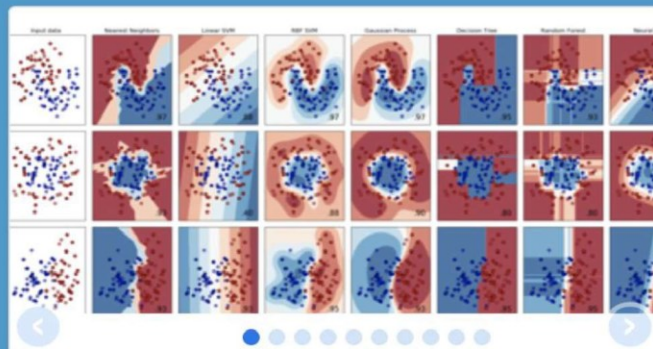
Is important that you are familiar with python and statistics

You can load dataset using **pandas**. Is Built on **Numpy, Scipy** & **Matplotlib**. There is a huge documentation on scikit-learn.org

Install it:

conda install -c anaconda scikit-learn





scikit-learn

Machine Learning in Python

- Simple and efficient tools for data mining and data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying to which category an object belongs to.

Applications: Spam detection, Image recognition.

Algorithms: SVM, nearest neighbors, random forest, ... [— Examples](#)

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.

Algorithms: SVR, ridge regression, Lasso, ... [— Examples](#)

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, spectral clustering, mean-shift, ... [— Examples](#)

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency

Algorithms: PCA, feature selection, non-negative matrix factorization. [— Examples](#)

Model selection

Comparing, validating and choosing parameters and models.

Goal: Improved accuracy via parameter tuning

Modules: grid search, cross validation, metrics. [— Examples](#)

Preprocessing

Feature extraction and normalization.

Application: Transforming input data such as text for use with machine learning algorithms.

Modules: [Text processing](#), [feature extraction](#)

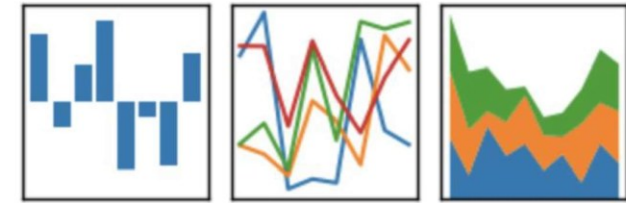


UNIVERSITÀ
DEGLI STUDI
DI SALERNO



data manipulation pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



Pandas is used to manage data. It works with dataframes (strings of numerical data). Here we will see:

- how to load csv file in Pandas
- how to use prebuilt datasets, to help you testing scikit-learn algorithms (you'll never use this kind of dataset but they are useful indeed)

Install it:

conda install -c anaconda pandas



UNIVERSITÀ
DEGLI STUDI
DI SALERNO



6.2. Toy datasets

scikit-learn comes with a few small standard datasets that do not require to download any file from some external website.

They can be loaded using the following functions:

<code>load_boston</code> ([return_X_y])	Load and return the boston house-prices dataset (regression).
<code>load_iris</code> ([return_X_y])	Load and return the iris dataset (classification).
<code>load_diabetes</code> ([return_X_y])	Load and return the diabetes dataset (regression).
<code>load_digits</code> ([n_class, return_X_y])	Load and return the digits dataset (classification).
<code>load_linnerud</code> ([return_X_y])	Load and return the linnerud dataset (multivariate regression).
<code>load_wine</code> ([return_X_y])	Load and return the wine dataset (classification).
<code>load_breast_cancer</code> ([return_X_y])	Load and return the breast cancer wisconsin dataset (classification).

These datasets are useful to quickly illustrate the behavior of the various algorithms implemented in scikit-learn. They are however often too small to be representative of real world machine learning tasks.

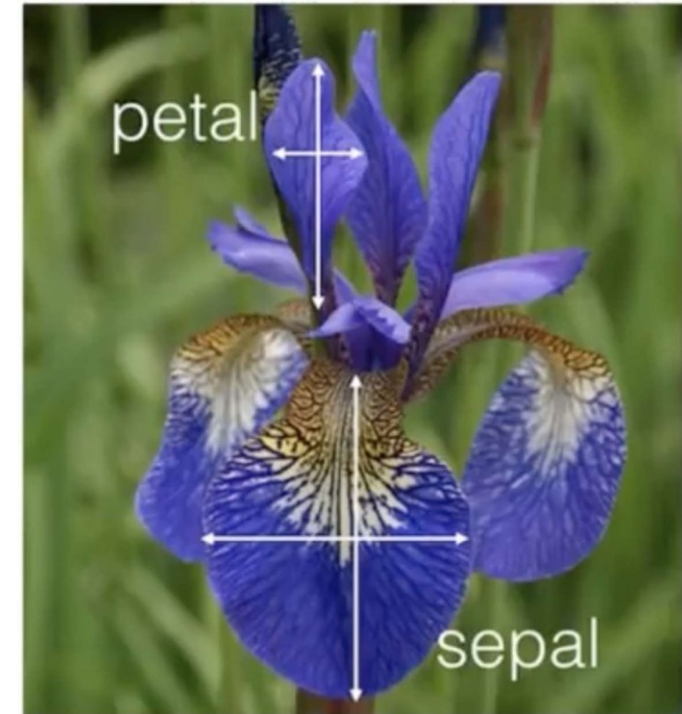


UNIVERSITÀ
DEGLI STUDI
DI SALERNO



IRIS Dataset

- is a famous example dataset
- the dataset consists of 50 samples from three species of Iris (**Setosa**, **Virginica** and **Versicolor**)
- Four features were measured from each samples
(**sepals lenght**, **sepals width**, **petal lenght** and **petal width**) in centimeters.



UNIVERSITÀ
DEGLI STUDI
DI SALERNO



IRIS Dataset



Iris Versicolor



Iris Setosa



Iris Virginica



UNIVERSITÀ
DEGLI STUDI
DI SALERNO







Jupyter notebook and Jupyter-Lab



Jupyter-Lab and Notebook

- Notebook documents (or “notebooks”, all lower case) are documents produced by the [Jupyter Notebook App](#), which contain both computer code (e.g. python) and rich text elements (paragraph, equations, figures, links, etc...). Notebook documents are both human-readable documents containing the analysis description and the results (figures, tables, etc..) as well as executable documents which can be run to perform data analysis.
- **JupyterLab: A Next-Generation Notebook Interface**
 - JupyterLab is the latest web-based interactive development environment for notebooks, code, and data. Its flexible interface allows users to configure and arrange workflows in data science, scientific computing, computational journalism, and machine learning. A modular design invites extensions to expand and enrich functionality.
- **Jupyter Notebook: The Classic Notebook Interface**
 - The Jupyter Notebook is the original web application for creating and sharing computational documents. It offers a simple, streamlined, document-centric experience.

Installing Jupyter

JupyterLab

Install JupyterLab with **pip**:

```
pip install jupyterlab
```

Note: If you install JupyterLab with conda or mamba, we recommend using **the conda-forge channel**.

Once installed, launch JupyterLab with:

```
jupyter-lab
```

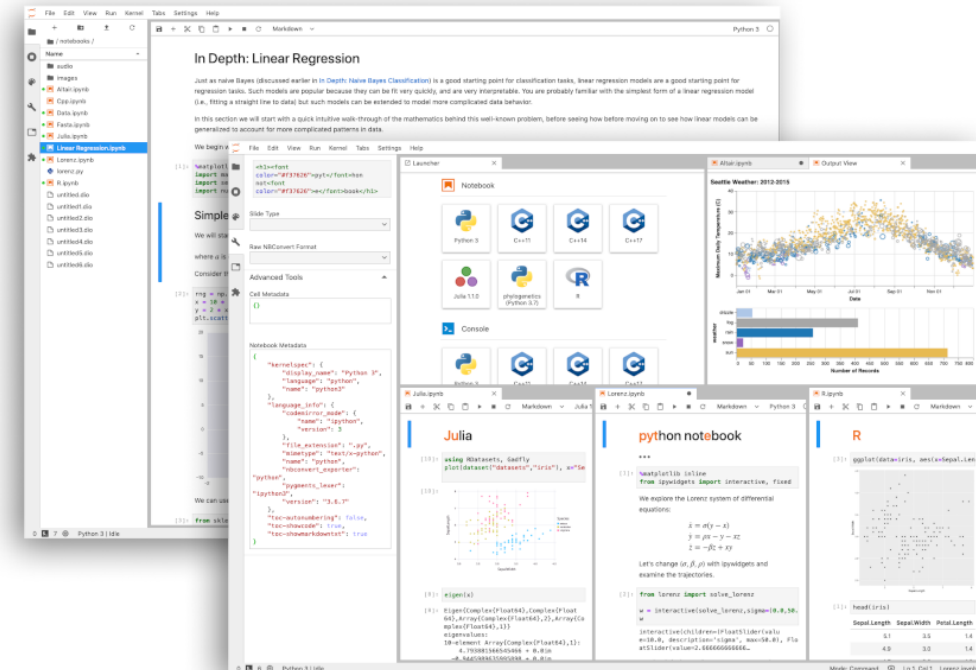
Jupyter Notebook

Install the classic Jupyter Notebook with:

```
pip install notebook
```

To run the notebook:

```
jupyter notebook
```



Jupyter Markdown commands

You can use Markdown to format documentation you add to *Markdown cells* in your Jupyter notebook.

Here's how to format Markdown cells in Jupyter notebooks:

Headings

Use the number sign (#) followed by a blank space for notebook titles and section headings:

- `#` for titles
- `##` for major headings
- `###` for subheadings
- `####` for 4th level subheadings

Emphasis

Use the following code to emphasize text:

- Bold text: `__string__` or `**string**`
- Italic text: `_string_` or `*string*`

```
# for titles
## for major headings
### for subheadings
#### for 4th level subheadings
```

Bold text: `__string__` or `**string**`

Italic text: `_string_` or `*string*`



for titles

for major headings

for subheadings

for 4th level subheadings

Bold text: **string** or **string**

Italic text: *string* or *string*

Jupyter Markdown commands

- **Mathematical symbols**
 - Surround mathematical symbols with a dollar sign (\$), for example:
 - $\$ \text{ mathematical symbols } \$$
- **Monospace font**
 - Surround text with a grave accent (`), ALT+96) also called a back single quotation mark, for example:
 - ``string``
 - You can use the monospace font for file paths, file names, message text that users see, or text that users enter.
- **Line breaks**
 - Sometimes markdown doesn't make line breaks when you want them. To force a linebreak, use the following code: `<br>`
- **Indenting**
 - Use the greater than sign (>) followed by a space, for example:
 - `> Text that will be indented when the Markdown is rendered.`

```
__Mathematical symbols__  
$ mathematical symbols $  
  
__Monospace font__  
`string`  
  
__Line breaks__  
line1<br>  
line2  
  
__Indenting__  
> Text that will be indented when the Markdown is rendered.  
  
Non indented text
```



Mathematical symbols *mathematicalsymbols*

Monospace font `string`

Line breaks line1
line2

Indenting

Text that will be indented when the Markdown is rendered.

Non indented text

Jupyter Markdown commands

- **Bullets**

- To create a circular bullet point, use one of the following methods. Each bullet point must be on its own line.
- A hyphen (-) followed by one or two spaces, for example:
 - - Bulleted item
- A space, a hyphen (-) and a space, for example:
 - - Bulleted item
- An asterisk (*) followed by one or two spaces, for example:
 - * Bulleted item
- To create a sub bullet, press Tab before entering the bullet point using one of the methods described above. For example:
 - - Main bullet point
 - - Sub bullet point

- **Numbered lists**

- To create a numbered list, enter 1. followed by a space, for example:
 - 1. Numbered item
 - 1. Numbered item

Bullets

To create a circular bullet point, use one of the following methods. Each bullet point must be on its own line. A hyphen (-) followed by one or two spaces, for example:

- Bulleted item

A space, a hyphen (-) and a space, for example:

- Bulleted item

An asterisk (*) followed by one or two spaces, for example:

* Bulleted item

To create a sub bullet, press Tab before entering the bullet point using

- Main bullet point

- Sub bullet point

Numbered lists

To create a numbered list, enter 1. followed by a space, for example:

1. Numbered item

1. Numbered item

Bullets To create a circular bullet point, use one of the following methods. Each bullet point must be on its own line. For example:

- Bulleted item A space, a hyphen (-) and a space, for example:
- Bulleted item An asterisk (*) followed by one or two spaces, for example:
- Bulleted item To create a *sub bullet*, press Tab before entering the bullet point using
- Main bullet point
 - Sub bullet point

Numbered lists To create a numbered list, enter 1. followed by a space, for example

1. Numbered item

2. Numbered item

Jupyter Markdown commands

- **Graphics**

- You can attach image files directly to a notebook in Markdown cells by dragging and dropping it into the cell.
- To add images to other cell types, use graphics that are hosted on the web with this code, substituting url/name with the full URL and name of the image:
- ``
- Restriction: You cannot add captions to graphics.

- **Geometric shapes**

- Use `&#` followed by the decimal or hex reference number for the shape, for example:
 - `&#reference_number`
 - For a list of reference numbers, see UTF-8 Geometric shapes: https://www.w3schools.com/charsets/ref_utf_geometric.asp

- **Horizontal lines**

- On a new line, enter three asterisks:
- `***`

Char	Dec	Hex	▯	9643	25AB
■	9632	25A0	▬	9644	25AC
□	9633	25A1	▭	9645	25AD
▢	9634	25A2	▣	9646	25AE
▣	9635	25A3	▤	9647	25AF
▥	9636	25A4	▦	9648	25B0
▧	9637	25A5	▨	9649	25B1
▩	9638	25A6	▴	9650	25B2
▪	9639	25A7	▵	9651	25B3
▫	9640	25A8	▴	9652	25B4
▬	9641	25A9	▴	9653	25B5
▭	9642	25AA	▴	9654	25B6

`■` `□` `▢` `▣`



Jupyter Markdown commands

- A table can be constructed using | (pipe symbol) and — (dash) to mark columns and rows.
- **Markdown Syntax**

```
| Header | Header | Header |
```

```
| ----- | ----- | ----- |
```

```
| A      | B      | C      |
```

```
| X      | Y      | Z      |
```

Equivalent HTML Syntax

```
<table>
<thead>
<tr><th>Header</th><th>Header</th><th>Header</th></tr>
</thead>
<tbody>
<tr><td>A</td><td>B</td><td>C</td></tr>
<tr><td>X</td><td>Y</td><td>Z</td></tr>
</tbody>
</table>
```

Header	Header	Header
A	B	C
X	Y	Z



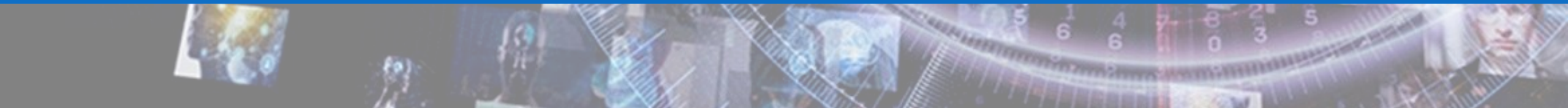
Header	Header	Header
A	B	C
X	Y	Z

Other sources

- Markdown options and commands are numerous.
- For further references:
 - <https://jupyter-notebook.readthedocs.io/en/stable/examples/Notebook/Working%20With%20Markdown%20Cells.html>
 - <https://medium.com/analytics-vidhya/the-ultimate-markdown-guide-for-jupyter-notebook-d5e5abf728fd>
 - <https://www.ibm.com/docs/en/watson-studio-local/1.2.3?topic=notebooks-markdown-jupyter-cheatsheet>
 - <https://www.datacamp.com/tutorial/markdown-in-jupyter-notebook>



The basis of Python programming



What is Python?

- Python is an interpreted, object-oriented, high-level programming language with dynamic semantics.
- Its high-level built in data structures, combined with dynamic typing and dynamic binding, make it very attractive for Rapid Application Development
- Python is simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance.
- Python supports modules and packages, which encourages program modularity and code reuse.
- The Python interpreter and the extensive standard library are available in source or binary form without charge for all major platforms and can be freely distributed.

Python keyword

- Keywords are predefined, reserved words used in Python programming that have special meanings to the compiler.
- We cannot use a keyword as a variable name, function name, or any other identifier. They are used to define the syntax and structure of the Python language.
- All the keywords except True, False and None are in lowercase and they must be written as they are.
- The list of all the keywords is given:

Python Keywords List				
False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Python datatypes

- Since everything is an object in Python programming, data types are actually classes and variables are instances(object) of these classes.
- **Integers**, floating-point numbers and complex numbers fall under Python numbers category. They are defined as int, float and complex classes in Python.
- **int** - holds signed integers of non-limited length.
- **float** - holds floating decimal points and it's accurate up to 15 decimal places.
- **complex** - holds complex numbers.
- We can use the type() function to know which class a variable or a value belongs to.

Data Types	Classes	Description
Numeric	int, float, complex	holds numeric values
String	str	holds sequence of characters
Sequence	list, tuple, range	holds collection of items
Mapping	dict	holds data in key-value pair form
Boolean	bool	holds either <code>True</code> or <code>False</code>
Set	set, frozenset	hold collection of unique items

Python datatypes

- **Python List Data Type**

- List is an ordered collection of similar or different types of items separated by commas and enclosed within brackets []. For example,

- `languages = ["Swift", "Java", "Python"]`

- **Python Tuple Data Type**

- Tuple is an ordered sequence of items same as a list. The only difference is that tuples are immutable. Tuples once created cannot be modified.

- In Python, we use the parentheses () to store items of a tuple. For example,

- `product = ('Xbox', 499.99)`

- **Access Tuple Items**

- Similar to lists, we use the index number to access tuple items in Python.

Python datatypes

- **Python String Data Type**

- String is a sequence of characters represented by either single or double quotes.

- **Python Set Data Type**

- Set is an unordered collection of unique items. Set is defined by values separated by commas inside braces {}

- **Python Dictionary Data Type**

- Python dictionary is an ordered collection of items. It stores elements in key/value pairs. Each pair key/value separated by comma and in each pair the key and value are separated by colon.

Python datatypes, examples

Numeric

```
num1 = 5
print(num1, 'is of type', type(num1))
num2 = 2.0
print(num2, 'is of type', type(num2))
num3 = 1+2j
print(num3, 'is of type', type(num3))
```

List

```
languages = ["Swift", "Java", "Python"]
# access element at index 0
print(languages[0]) # Swift
# access element at index 2
print(languages[2]) # Python
```

Tuple

```
# create a tuple
product = ('Microsoft', 'Xbox', 499.99)
# access element at index 0
print(product[0])
# Microsoft # access element at index 1
print(product[1]) # Xbox
```

String

```
name = 'Python'
print(name)
message = 'Python for beginners'
print(message)
```

Set

```
# create a set named student_id
student_id = {112, 114, 116, 118, 115}
# display student_id elements
print(student_id)
# display type of student_id print(type(student_id))
```

Dictionary

```
# create a dictionary named capital_city
capital_city = {'Nepal': 'Kathmandu', 'Italy': 'Rome',
                'England': 'London'}
print(capital_city['Nepal']) # prints Kathmandu
print(capital_city['Kathmandu']) # throws error message
```

Python I/O

- In Python, we can simply use the *print()* function to print output.
- While programming, we might want to take the input from the user. In Python, we can use the *input()* function.

```
# using input() to take user input
num = input('Enter a number: ')
print('You Entered:', num)
print('Data type of num:', type(num))
```

Python Operators

Arithmetic operator

Operator	Operation	Example
<code>+</code>	Addition	<code>5 + 2 = 7</code>
<code>-</code>	Subtraction	<code>4 - 2 = 2</code>
<code>*</code>	Multiplication	<code>2 * 3 = 6</code>
<code>/</code>	Division	<code>4 / 2 = 2</code>
<code>//</code>	Floor Division	<code>10 // 3 = 3</code>
<code>%</code>	Modulo	<code>5 % 2 = 1</code>
<code>**</code>	Power	<code>4 ** 2 = 16</code>

Comparison operator

Operator	Meaning	Example
<code>==</code>	Is Equal To	<code>3 == 5</code> gives us False
<code>!=</code>	Not Equal To	<code>3 != 5</code> gives us True
<code>></code>	Greater Than	<code>3 > 5</code> gives us False
<code><</code>	Less Than	<code>3 < 5</code> gives us True
<code>>=</code>	Greater Than or Equal To	<code>3 >= 5</code> give us False
<code><=</code>	Less Than or Equal To	<code>3 <= 5</code> gives us True

Assignment operators

Operator	Name	Example
<code>=</code>	Assignment Operator	<code>a = 7</code>
<code>+=</code>	Addition Assignment	<code>a += 1 # a = a + 1</code>
<code>-=</code>	Subtraction Assignment	<code>a -= 3 # a = a - 3</code>
<code>*=</code>	Multiplication Assignment	<code>a *= 4 # a = a * 4</code>
<code>/=</code>	Division Assignment	<code>a /= 3 # a = a / 3</code>
<code>%=</code>	Remainder Assignment	<code>a %= 10 # a = a % 10</code>
<code>**=</code>	Exponent Assignment	<code>a **= 10 # a = a ** 10</code>

Logical operator

Operator	Example	Meaning
<code>and</code>	<code>a and b</code>	Logical AND: <code>True</code> only if both the operands are <code>True</code>
<code>or</code>	<code>a or b</code>	Logical OR: <code>True</code> if at least one of the operands is <code>True</code>
<code>not</code>	<code>not a</code>	Logical NOT: <code>True</code> if the operand is <code>False</code> and vice-versa.

Python: other operators

- Bitwise operators act on operands as if they were strings of binary digits. They operate bit by bit, hence the name.
 - For example, 2 is 10 in binary and 7 is 111.
 - In the table below: Let x = 10 (0000 1010 in binary) and y = 4 (0000 0100 in binary)

Operator	Meaning	Example
&	Bitwise AND	x & y = 0 (0000 0000)
	Bitwise OR	x y = 14 (0000 1110)
~	Bitwise NOT	~x = -11 (1111 0101)
^	Bitwise XOR	x ^ y = 14 (0000 1110)
>>	Bitwise right shift	x >> 2 = 2 (0000 0010)
<<	Bitwise left shift	x << 2 = 40 (0010 1000)

In Python, *is* and *is not* are used to check if two values are located on the same part of the memory. Two variables that are equal does not imply that they are identical.

Operator	Meaning	Example
is	True if the operands are identical (refer to the same object)	x is True
is not	True if the operands are not identical (do not refer to the same object)	x is not True

in and *not in* are the membership operators. They are used to test whether a value or variable is found in a sequence (string, list, tuple, set and dictionary).

Operator	Meaning	Example
in	True if value/variable is found in the sequence	5 in x
not in	True if value/variable is not found in the sequence	5 not in x

is and is not

```
x1 = 5
y1 = 5
x2 = 'Hello'
y2 = 'Hello'
x3 = [1,2,3]
y3 = [1,2,3]
print(x1 is y1) # prints True
print(x2 is y2) # prints True
print(x3 is y3) # prints False
```

```
x = 'Hello world'
y = {1:'a', 2:'b'}
# check if 'H' is present in x string
print('H' in x) # prints True
# check if 'hello' is present in x string
print('hello' not in x) # prints True
# check if '1' key is present in y
print(1 in y) # prints True
# check if 'a' key is present in y print('a' in y) #
prints False
```

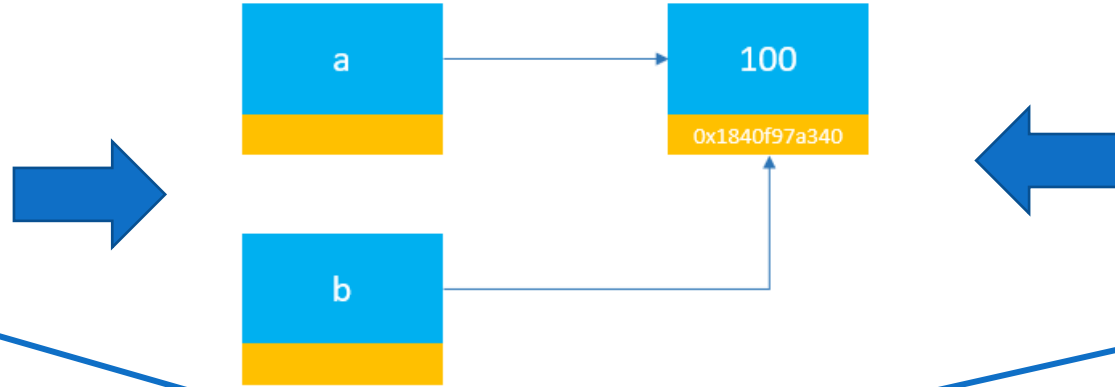
- Two variables can be equal but not identical.
- The Python memory manager resuses in some cases the used memory.
- This is true for variable and strings (*immutable objects*) but not for lists that are *mutable objects*
- Then:
 - Use the **is** operator to check if two variables reference the same object.
 - Use the **is** operator to check two variables for identity and **==** to check for two variables for equality.

Immutable and mutable objects

Immutable

```
a = 100  
b = a
```

```
result = a is b  
print(result)
```



```
a = 100  
b = 100
```

```
result = a is b  
print(result)
```

True

Mutable

```
ranks = [1, 2, 3]  
rates = [1, 2, 3]
```

```
result = ranks is rates  
print(result)
```



```
ranks = [1, 2, 3]  
rates = [1, 2, 3]
```

```
is_equal = (ranks == rates)
```

```
print(is_equal)
```



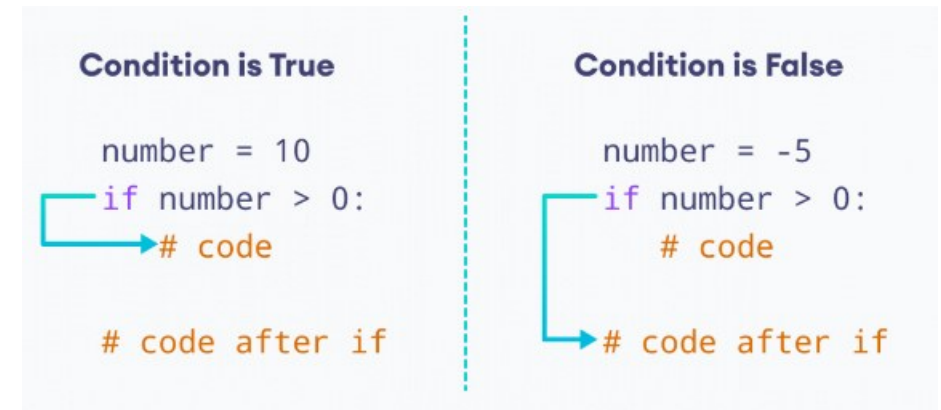
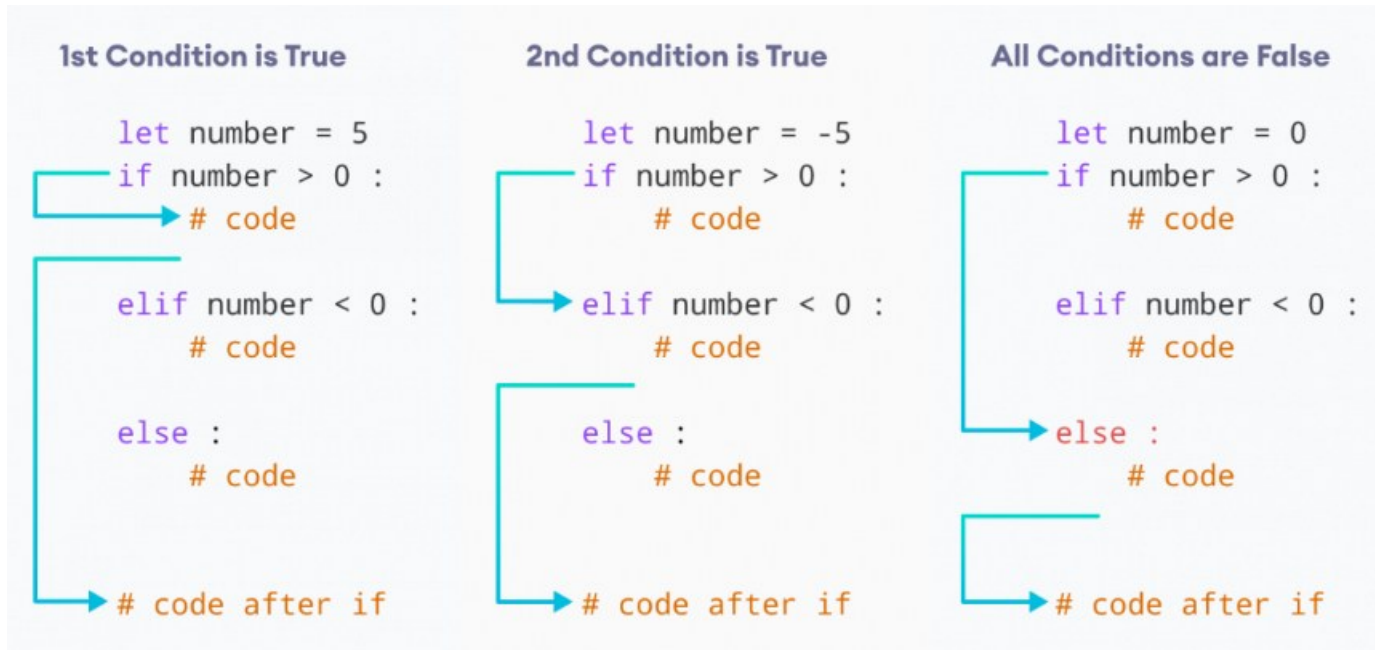
False

True

If statement

In computer programming, we use the if statement to run a block code only when a certain condition is met. For example, assigning grades (**A, B, C**) based on marks obtained by a student.

1. if the percentage is above **90**, assign grade **A**
2. if the percentage is above **75**, assign grade **B**
3. if the percentage is above **65**, assign grade **C**



```
number = 0
if number > 0:
    print("Positive number")
elif
    number == 0:
    print('Zero')
else:
    print('Negative number')
print('This statement is always executed')
```

For loop

- In Python, the for loop is used to run a block of code for a certain number of times. It is used to iterate over any sequences such as list, tuple, string, etc.

```
for val in sequence:  
    # statement(s)
```

- Example

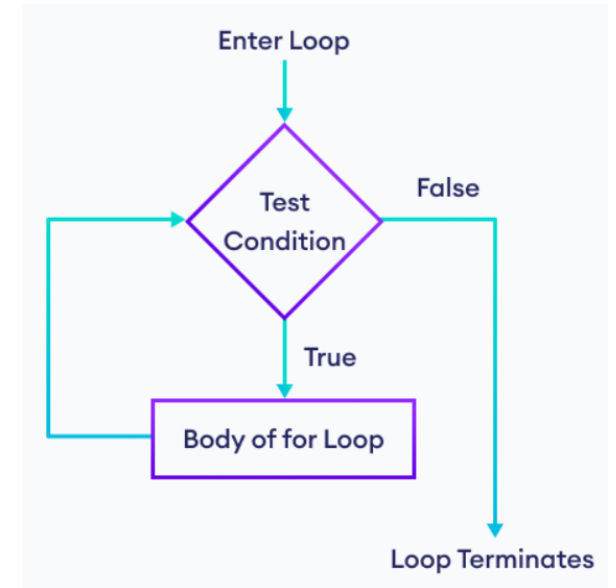
```
languages = ['Swift', 'Python', 'Go', 'JavaScript']  
# access items of a list using for loop  
for language in languages:  
    print(language)
```

```
# use of range() to define a range of  
values values = range(4)  
# iterate from i = 0 to i = 3  
for i in values:  
    print(i)
```

Python for loop with else

A for loop can have an optional else block as well. The else part is executed when the loop is finished.

```
digits = [0, 1, 5]  
  
for i in digits:  
    print(i)  
else:  
    print("No items left.")
```



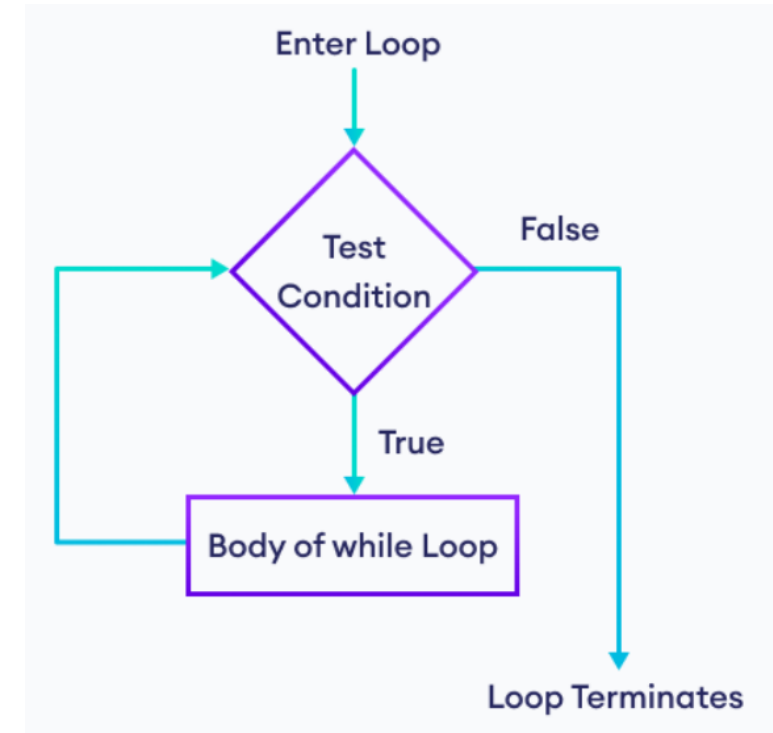
While loop

- Python while loop is used to run a block code until a certain condition is met.

```
while condition:  
    # body of while loop
```

- Example:

```
# program to calculate the sum of numbers  
# until the user enters zero  
  
total = 0  
  
number = int(input('Enter a number: '))  
  
# add numbers until number is zero  
while number != 0:  
    total += number    # total = total + number  
  
    # take integer input again  
    number = int(input('Enter a number: '))  
  
print('total =', total)
```



While loop also can have an optional else block

Functions

The syntax to declare a function is:

```
def function_name(arguments):  
    # function body  
  
    return
```

Here,

- `def` - keyword used to declare a function
- `function_name` - any name given to the function
- `arguments` - any value passed to function
- `return` (optional) - returns value from a function

EXAMPLE :

A diagram illustrating the syntax of a function definition. It shows the code: `def greet():` followed by a line of code (indicated by a blue arrow labeled '2' pointing to the line) and then `greet()` followed by another line of code (indicated by a blue arrow labeled '3' pointing to the line). A blue arrow labeled '1' points from the end of the second line of code back to the opening parenthesis of the `def` statement, representing the return path.

```
def greet():  
    print('Hello World!')
```

```
# call the function  
greet()
```

```
print('Outside function')
```

Passing arguments to functions

```
def add_numbers(a, b):  
    sum = a + b  
    print('Sum:', sum)
```

```
add_numbers(2, 3)
```

Output: Sum: 5

- In the above example, the function `add_numbers()` takes two parameters: `a` and `b`. Notice the line,
 - `add_numbers(2, 3)`
- Here, `add_numbers(2, 3)` specifies that parameters `a` and `b` will get values 2 and 3 respectively.

Passing arguments to functions

- In Python, we can provide **default** values to function arguments.
- We use the = operator to provide default values.

```
def add_numbers( a = 7, b = 8):  
    sum = a + b  
    print('Sum:', sum)
```

Output

```
Sum: 5  
Sum: 10  
Sum: 15
```

```
# function call with two arguments  
add_numbers(2, 3)
```

```
# function call with one argument  
add_numbers(a = 2)
```

```
# function call with no arguments  
add_numbers()
```

Passing arguments to functions

In keyword arguments, arguments are assigned based on the **name of arguments**. For example,

```
def display_info(first_name, last_name):  
    print('First Name:', first_name)  
    print('Last Name:', last_name)  
  
display_info(last_name = 'Cartman',  
             first_name = 'Eric')
```

Arbitrary arguments allow us to pass a varying number of values during a function call.

```
# program to find sum of multiple numbers
```

```
def find_sum(*numbers):  
    result = 0  
  
    for num in numbers:  
        result = result + num  
  
    print("Sum = ", result)
```

```
# function call with 3 arguments  
find_sum(1, 2, 3)
```

```
# function call with 2 arguments  
find_sum(4, 9)
```

Anonymous functions

- In Python, a lambda function is a special type of function without the function name.

lambda argument(s) : expression

argument(s) - any value passed to the lambda function

expression - expression is executed and returned

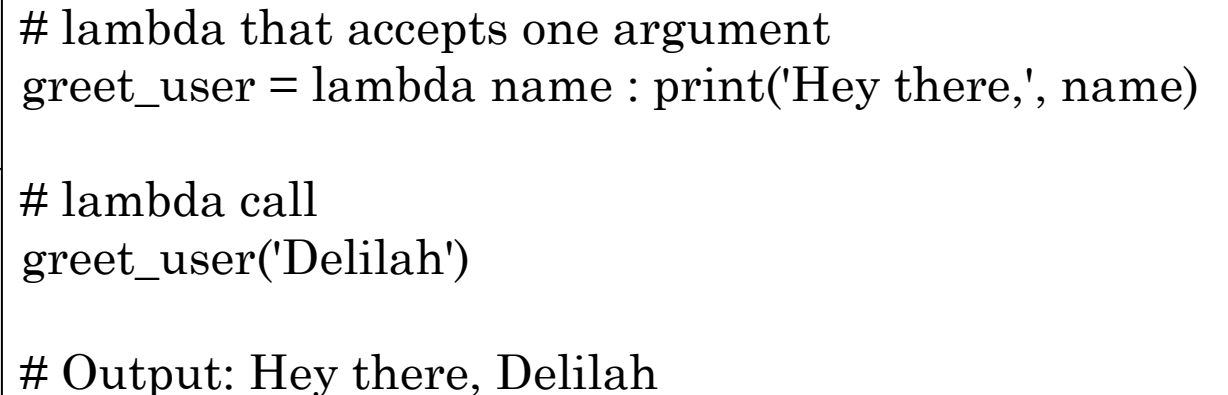
- Below, we have created a lambda function that prints 'Hello World':

greet = lambda : print('Hello World')

call the lambda

greet()

- Also arguments can be easily passed:



```
# lambda that accepts one argument
greet_user = lambda name : print('Hey there,', name)

# lambda call
greet_user('Delilah')

# Output: Hey there, Delilah
```

Global, Local and Non-local variables

- A variable **scope** specifies the region where we can access a variable.
- Based on the scope, we can classify Python variables into three types:
 1. Local Variables
 2. Global Variables
 3. Nonlocal Variables
- When we declare variables inside a function, these variables will have a *local scope*
- A variable declared outside of the function or in global scope is known as a *global variable*. This means that a global variable can be accessed inside or outside of the function.
 - *global* keyword can be used to place in the global scope a variable created locally.
- In Python, *nonlocal variables* are used in nested functions whose local scope is not defined. This means that the variable can be *neither in the local nor the global scope*.

Global, Local and Non-local variables

LOCAL

```
def greet():  
  
    # local variable  
    message = 'Hello'  
  
    print('Local', message)  
  
greet()  
  
# try to access message variable  
# outside greet() function  
print(message)
```

GLOBAL

```
# declare global variable  
message = 'Hello'  
  
def greet():  
    # declare local variable  
    print('Local', message)  
  
greet()  
print('Global', message)
```

NON-LOCAL

```
# outside function  
def outer():  
    message = 'local'  
  
    # nested function  
    def inner():  
  
        # declare nonlocal variable  
        nonlocal message  
  
        message = 'nonlocal'  
        print("inner:", message)  
  
    inner()  
    print("outer:", message)  
  
outer()
```

A little bit more about Global Variables

- If we try to modify the global variable from inside a function...
- **ERROR:**
 - *UnboundLocalError: local variable 'c' referenced before assignment*
- This is because we can only access the global variable but cannot modify it from inside the function.
 - The solution for this is to use **the global keyword**.
- **The basic rules for global keyword in Python are:**
 - When we create a variable inside a function, it is local by default.
 - When we define a variable outside of a function, it is global by default. You don't have to use the global keyword.
 - We use the global keyword to read and write a global variable inside a function.
 - Use of the global keyword outside a function has no effect.

```
# global variable  
c = 1
```

```
def add():
```

```
    # use of global keyword  
    global c
```

```
    # increment c by 2  
    c = c + 2
```

```
    print(c)
```

```
add()
```


Modules

- As our program grows bigger, it may contain many lines of code.
- Instead of putting everything in a single file, we can use modules to separate codes in separate files as per their functionality.
- This makes our code organized and easier to maintain.
- **Module** is a file that contains code to perform a *specific task*. A module may contain variables, functions, classes etc.
- We use the import keyword to do this. To import our previously defined module example, we type the following in the Python prompt.

```
# Python Module addition
```

```
def add(a, b):
```

```
    result = a + b  
    return result
```

```
import example
```

```
example.add(4,5) # returns 9
```

Import Python Standard Library Modules

- The Python standard library contains well over 200 modules. We can import a module according to our needs.
- Suppose we want to get the value of π , first we import the math module and use *math.pi*.

```
# import standard math module
import math
# use math.pi to get value of pi
print("The value of pi is", math.pi)
```

- In Python, we can also import a module by renaming it.

```
# import module by renaming it
import math as m

print(m.pi)

# Output: 3.141592653589793
```

Python from...import statement

- We can import specific names from a module without importing the module as a whole.

```
# import only pi from math module
from math import pi
print(pi)
# Output: 3.141592653589793
```

- In Python, we can import all names(definitions) from a module

```
# import all names from the standard module math
from math import *
print("The value of pi is", pi)
```



C.A.S.A. Course

CONTEXT AWARE SECURITY ANALYTICS IN COMPUTER VISION

proff. Michele Nappi, Fabio Narducci and Carmen Bisogni

aa.2024/2025

